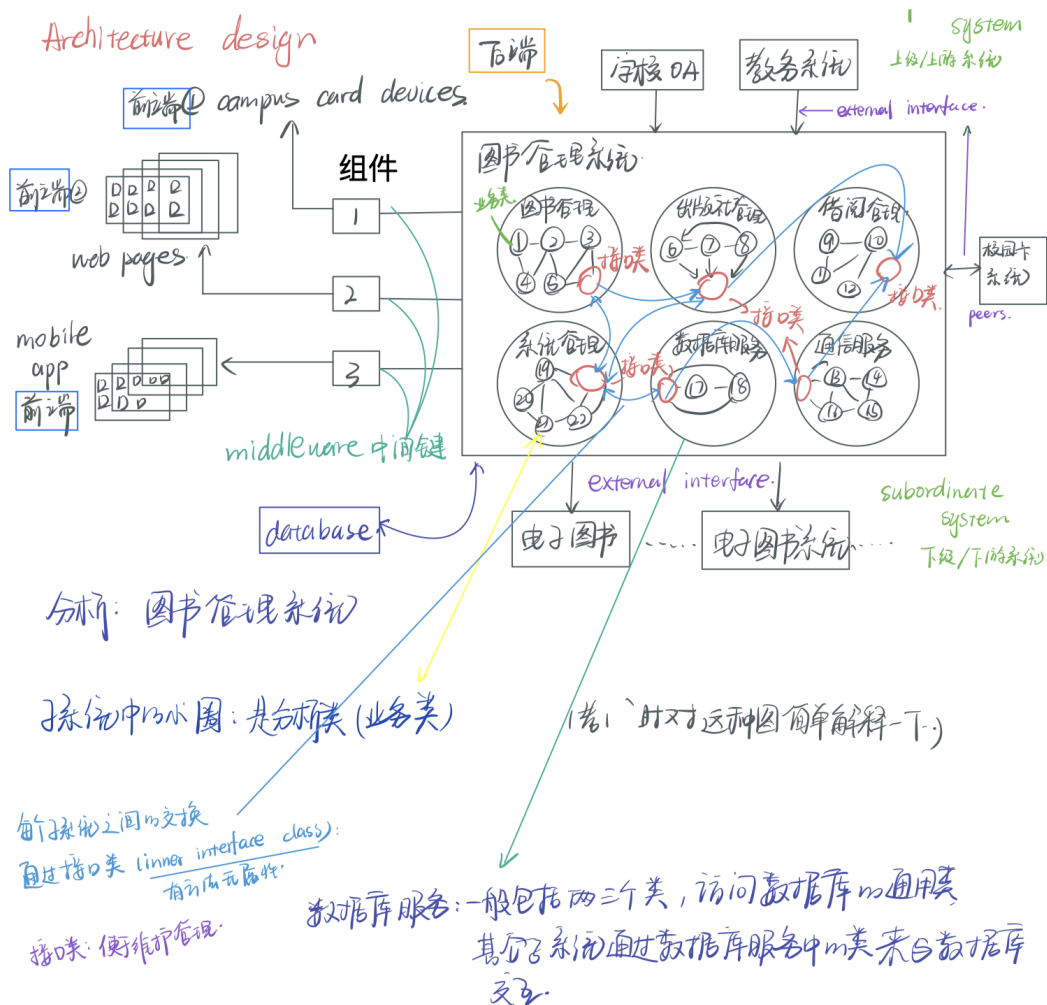


Chapter 13 概要设计Architectural Design

架构设计从数据建模开始（以分析类图为依据）

1. 构建子系统
2. 子系统内部的类，有什么类需要交互
3. 子系统之间的协作
4. 子系统之间的控制接口类

Overview: 一个示例



superordinate system 上游系统

subordinate system 下游系统

若要加入一个报数子系统

→ 使用数据流架构 (一个系统架构中可能有多种类型的架构)

以交易的数据(每天/月/年)为基础; 收集 $\rightarrow$ 处理 $\rightarrow$ 统计分析
 $\rightarrow$ 4 个步骤.



13.1 体系架构Software Architecture

13.1.1 什么是体系架构

架构是否有效，设计是否依据了需求规约的Use Case图和需求分析规约中的Class Diagram图

架构的作用：

- 在满足其定下的需求时，分析设计的有效性
- 在设计改变仍然很容易时可以考虑架构的替代方案
- 减少与软件构建相关的风险

组件：可以是一个类的程序模块，也可能是数据库和中间层

13.1.2 体系架构设计的重要性Importance

- 提供了便于StakeHolders交流的表达
- 对后续组件设计和coding有深远意义
- 一个相对小的，容易理解，展现组件如何组合在一起工作的模型

13.1.3 架构描述

要求：架构描述要简洁和易于理解，因为它形成了协商的基础，特别是在确定系统边界的时候

13.1.4 架构决策Architectural Decisions

架构师会考虑多种备选方案，并最终确定最能满足需求的架构

架构设计模板

Chapter 13 ARCHITECTURAL DESIGN



Architecture Decision Description Template

Each major architectural decision can be documented for later review by stakeholders who want to understand the architecture description that has been proposed. The template presented in this sidebar is an adapted and abbreviated version of a template proposed by Tyree and Ackerman [Tyr05].

Design issue:	Describe the architectural design issues that are to be addressed.	Alternatives:	Briefly describe the architectural design alternatives that were considered and why they were rejected.
Resolution:	State the approach you've chosen to address the design issue.	Argument:	State why you chose the resolution over other alternatives.
Category:	Specify the design category that the issue and resolution address (e.g., data design, content structure, component structure, integration, presentation).	Implications:	Indicate the design consequences of making the decision. How will the resolution affect other architectural design issues? Will the resolution constrain the design in any way?
Assumptions:	Indicate any assumptions that helped shape the decision.	Related decisions:	What other documented decisions are related to this decision?
Constraints:	Specify any environmental constraints that helped shape the decision (e.g., technology standards, available patterns, project-related issues).	Related concerns:	What other requirements are related to this decision?
		Work products:	Indicate where this decision will be reflected in the architecture description.
		Notes:	Reference any team notes or other documentation that was used to make the decision.

13.2 体系架构的流派Genres（从业务角度分类）

需要体系架构的系统：人工智能的体系架构，通信系统，设备，金融，游戏，工业，法律，....

13.3 架构风格Architectural Styles (从technology角度分析)

一个Styles会描述系统中的

- 一系列组件components (比如：数据库，计算模型等)
- 组件中的协作所需的connectors (对系统内部的分系列的连接，子系统间的连接，后台与中间键的连接)
- 一些constraints，定义了组件如何集成为系统需要
- 语义图semantic models (语义模型) → 便于设计者理解系统的层次

13.3.1 架构的简单分类

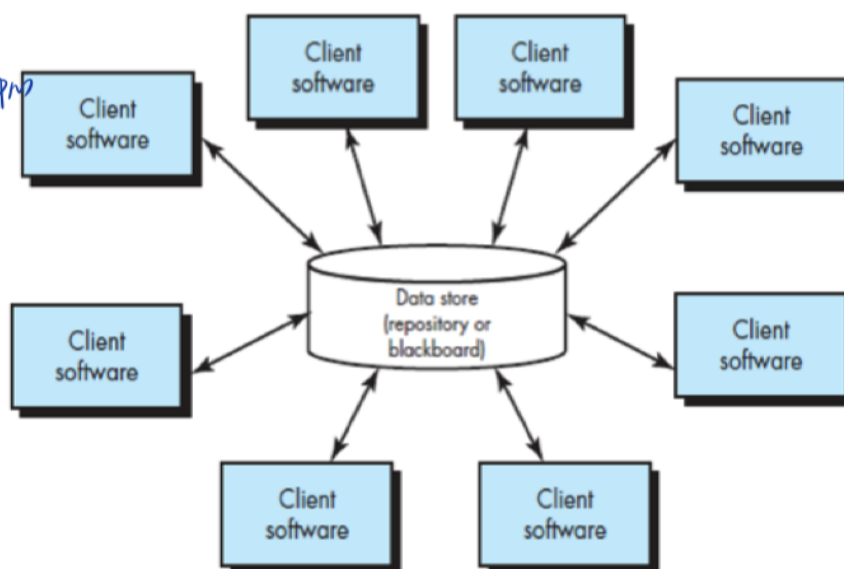
• Data-Centered Architectures:

数据为中心的软件体系结构:如Github等，多个客户端软件访问数据中心(最早)

数据存储在体系结构的中心，并经常被更新操作删除

Data-centered architecture

一种典型的以数据为中心的样式



• Data-Flow Architectures:

数据流的软件体系架构: 金融，银行的批处理系统batch subsystem

当输入数据要经过一系列计算操作组件转化为输入

模式：“管道-过滤器” (pipe-filter) 模式

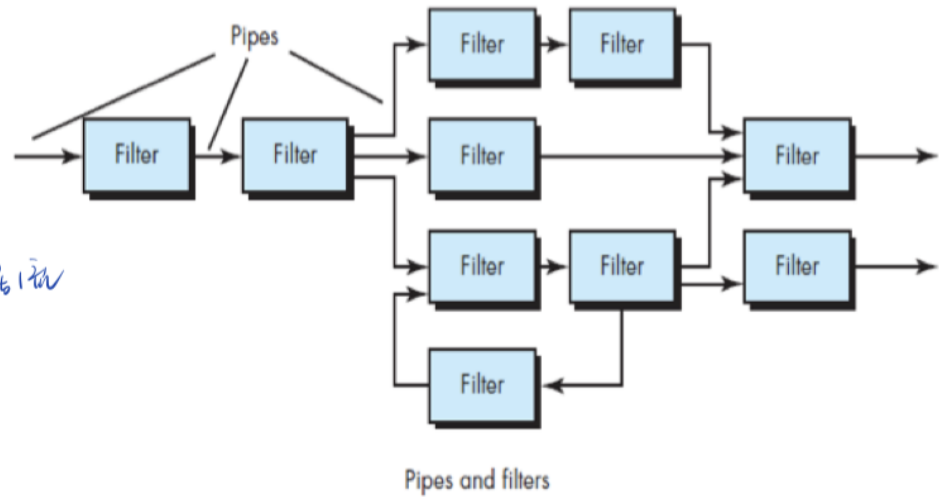
filters: 通过pipe连接，从一个组件传输到下一个组件,比如银行的批处理过滤出账户信息等

一般用c语言写

FIGURE 13.2

Data-flow architecture

是一个数据流



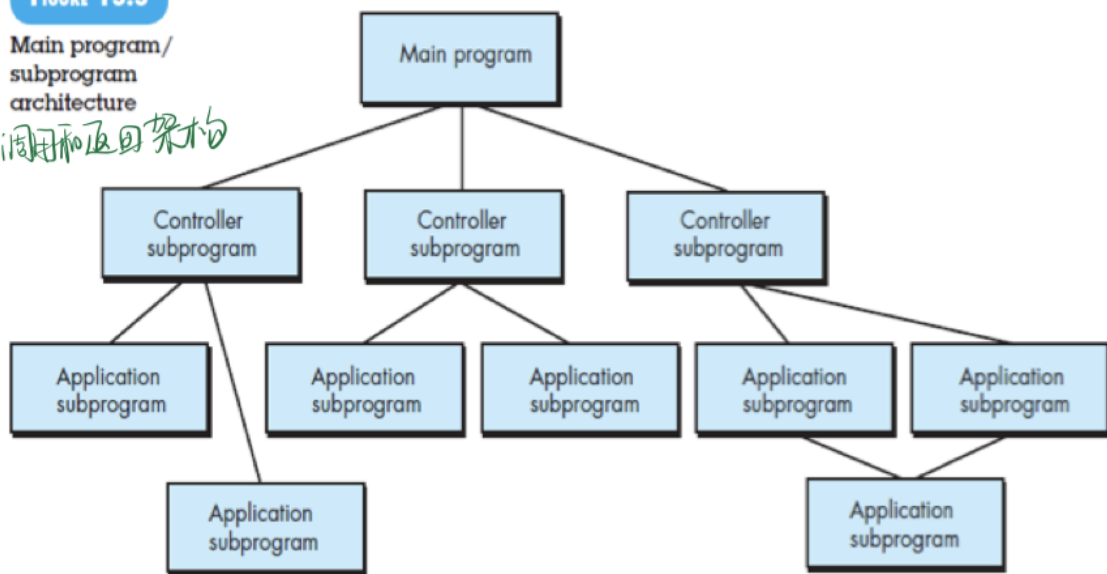
- **Call and Return Architectures** 调用和返回结构

C语言架构

FIGURE 13.3

Main program/
subprogram
architecture

调用和返回架构



Call And Return Architecture in C一般3个分支：

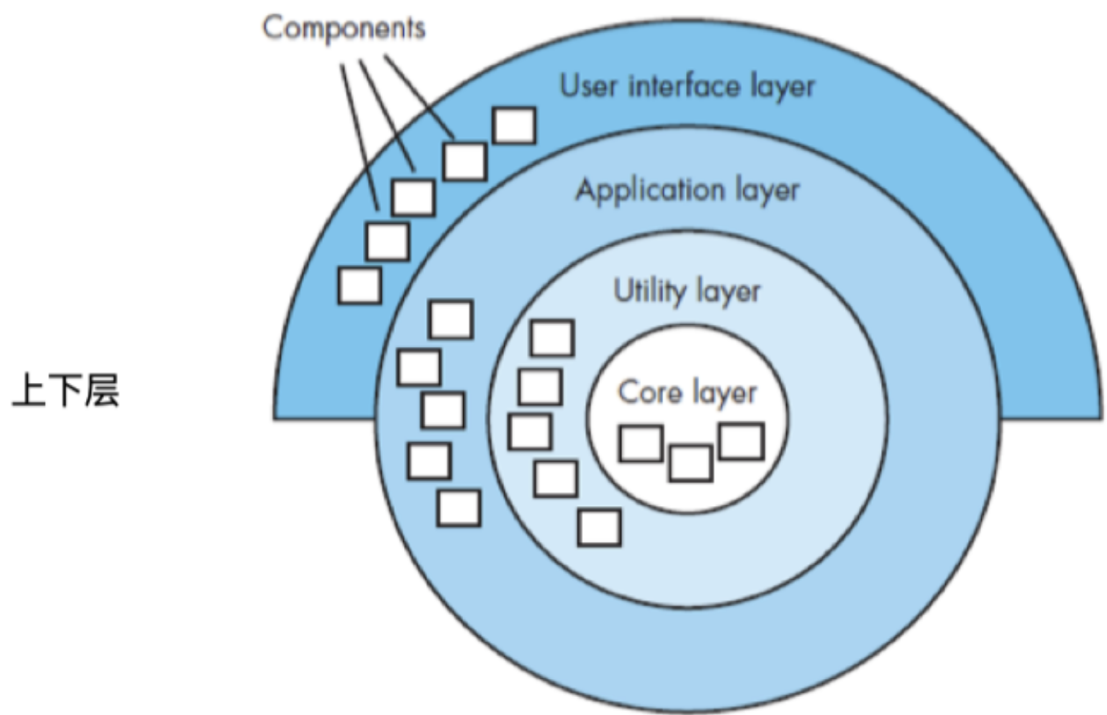
1. 最左侧分支为对输入处理
2. 中间分支为对业务本身的处理
3. 最右侧分支为对输出打印的处理么

- **Object-Oriented Architectures** 面向对象架构

设计以Use Case图和类图为依据，要用到CRC

- **Layered Architectures** 层次架构

典型：MVC



把前面几种进行整合



在这个银行系统中，左上是Data-Flow Arch, 左下是Call and Return Arch, 右下是Object-Oriented Arch, 可见复杂的系统中存在多种架构风格

13.3.2 架构模式Architectural Patterns

架构模式在特定上下文中以及一组限制和约束下解决特定于应用程序的问题。该模式提出了一种可以作为建筑设计的基础的架构解决方案。

13.3.3 架构的优化Organization and Refinement

建立一组设计标准很重要，它可用于评估衍生的架构设计。尤其是用敏捷开发时每一次都要注意架构的优化（特别是最初的迭代）

related: 演化过程模型(Chapter 4)，重构(Chapter 12)

13.6 系统架构Architectural Design 【很重要】

架构师要建立ACD(Architectural Context Diagram): 这个图就是系统的architecture(就是最上面的图)

包括:

1. 软件架构设计
2. 数据/数据库设计（以类图为依据，有时涉及数据字典设计，和接口紧密相关）
3. 接口设计（内部接口（类内方法互相调用），类间，子系统间方法调用，外部接口，用户接口）

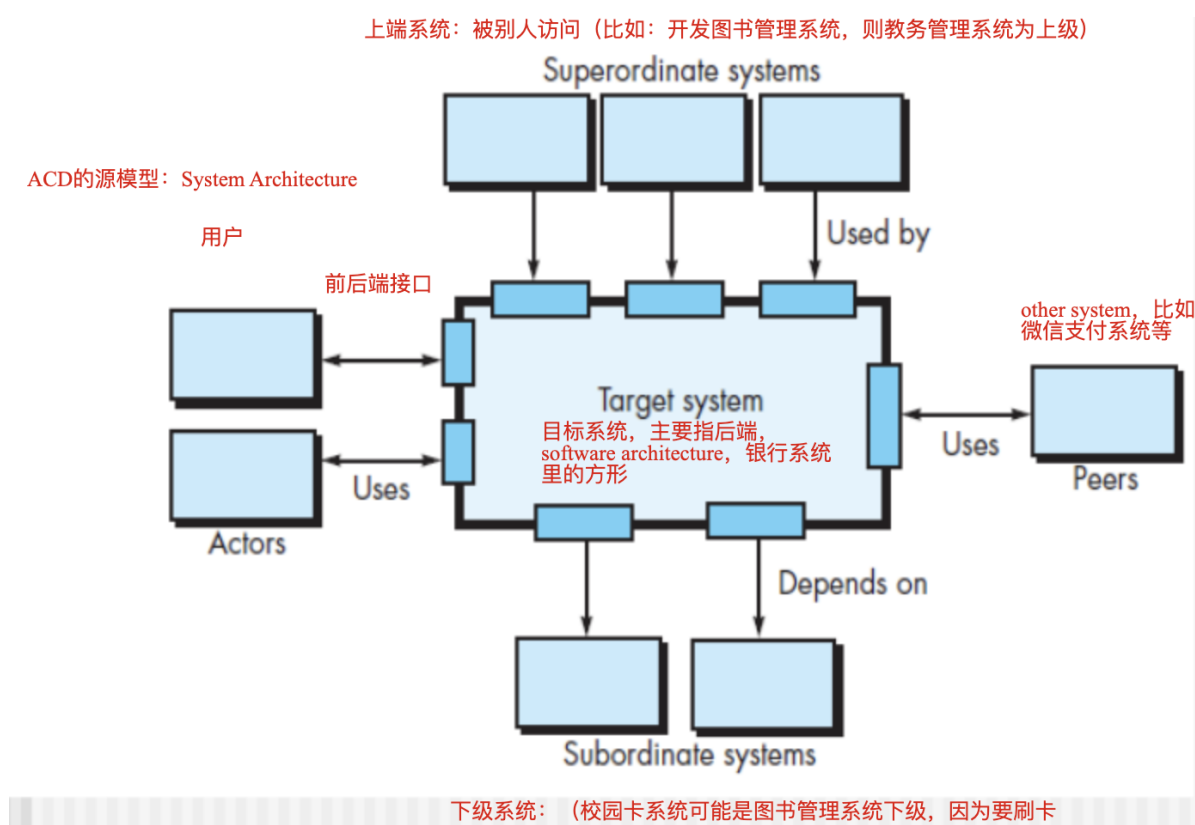
deployment diagram: 部署图，要包括（内网外网），系统的部署方式（云平台等）

(TODO)

13.6.1 系统上下文的建立

※ ACD图:

如何实现软件系统体系架构图？绘制ACD图后加入相应文字描述



内容：上游系统superordinate，下游系统subordinate，目标系统，Actors，Peers(同级系统,与目标系统同一个等级，与本系统进行交互，双方面通信)。

如何实现目标系统的体系结构设计：

分析类图—>设计类图—>设计类图细化加入控制类/数据库访问相关类—>构建子系统（依据use case diagram）—>完成目标系统软件体系架构

※以家庭安保系统为案例

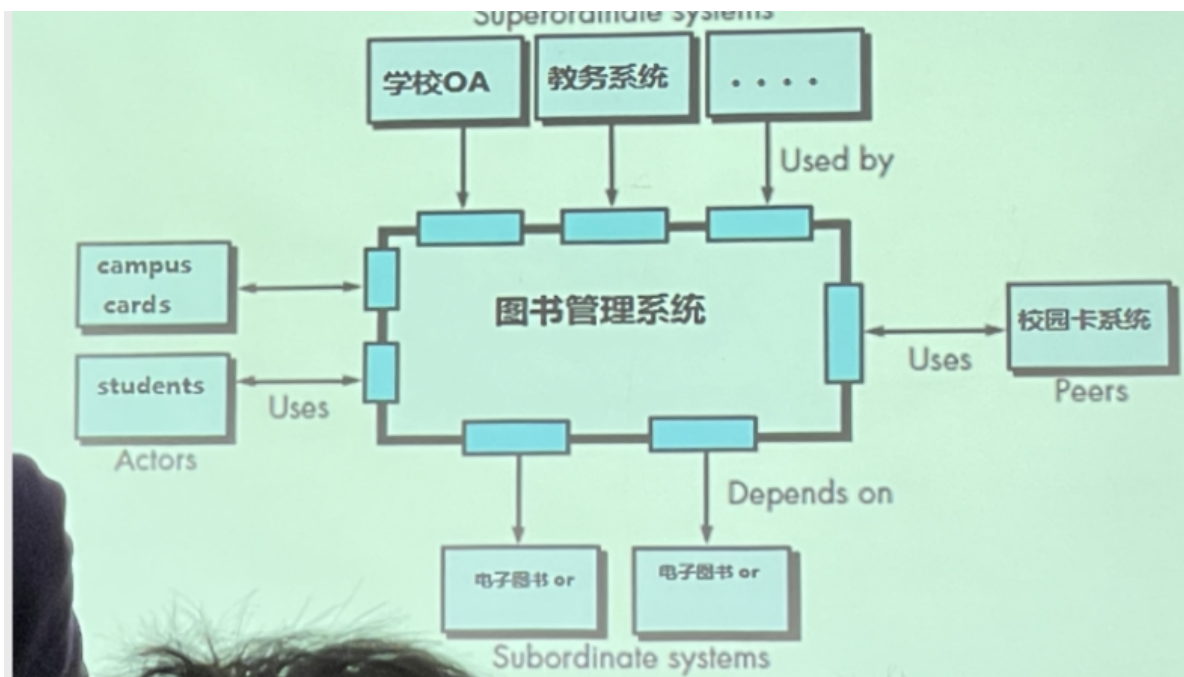
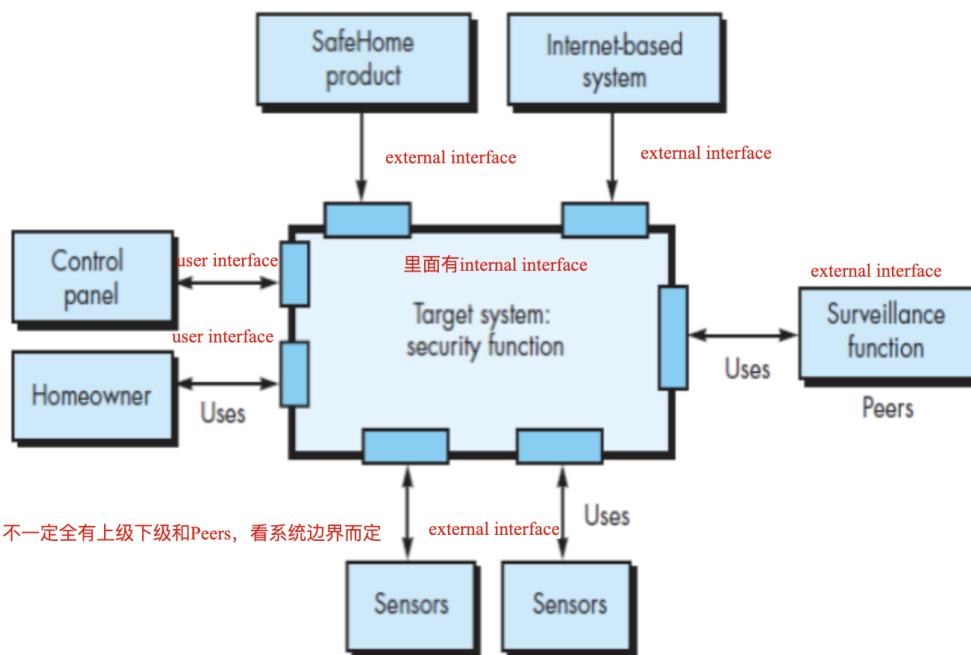


FIGURE 13.6

Architectural context diagram for the SafeHome security function



考试时，不一定全有上级下级和Peers，看系统边界而定